



Infor Mongoose Multi-Site Planning Guide

Important Notices

The material contained in this publication (including any supplementary information) constitutes and contains confidential and proprietary information of Infor.

By gaining access to the attached, you acknowledge and agree that the material (including any modification, translation or adaptation of the material) and all copyright, trade secrets and all other right, title and interest therein, are the sole property of Infor and that you shall not gain right, title or interest in the material (including any modification, translation or adaptation of the material) by virtue of your review thereof other than the non-exclusive right to use the material solely in connection with and the furtherance of your license and use of software made available to your company from Infor pursuant to a separate agreement, the terms of which separate agreement shall govern your use of this material and all supplemental related materials ("Purpose").

In addition, by accessing the enclosed material, you acknowledge and agree that you are required to maintain such material in strict confidence and that your use of such material is limited to the Purpose described above. Although Infor has taken due care to ensure that the material included in this publication is accurate and complete, Infor cannot warrant that the information contained in this publication is complete, does not contain typographical or other errors, or will meet your specific requirements.

As such, Infor does not assume and hereby disclaims all liability, consequential or otherwise, for any loss or damage to any person or entity which is caused by or relates to errors or omissions in this publication (including any supplementary information), whether such errors or omissions result from negligence, accident or any other cause.

Without limitation, U.S. export control laws and other applicable export and import laws govern your use of this material and you will neither export or re-export, directly or indirectly, this material nor any related materials or supplemental information in violation of such laws, or use such materials for any purpose prohibited by such laws.

Trademark Acknowledgements

The word and design marks set forth herein are trademarks and/or registered trademarks of Infor and/or related affiliates and subsidiaries. All rights reserved. All other company, product, trade or service names referenced may be registered trademarks or trademarks of their respective owners.

Publication Information

Release: Infor Mongoose 9.03

Publication date: October 24, 2016

Contents

About This Guide	5
Resulting Tools	5
Planning Worksheet	6
Chapter 1 What Is Your Corporate Structure?	9
Think About This... ..	9
Do This... ..	9
1. Create a Flowchart Showing Your Hierarchy	9
2. Create a Site Spreadsheet	9
Background Information	10
Chapter 2 How Many Sites Do You Need?	11
Think About This... ..	11
Do This... ..	11
Update the Flowchart	12
Update the Site Spreadsheet	12
Background Information	12
Physical Requirements	12
Chapter 3 Multi-Site per Database, or One Site per Database?	13
Think About This... ..	13
Do This... ..	16

Decide Whether You Need Multiple Sites in One Database	16
Background Information	16
Chapter 4 Can You Use the Same Forms Database for Multiple Sites?	19
Think About This...	19
Do This...	20
Update the Site Spreadsheet	20
Background Information	20
Chapter 5 Can You Use the Same Objects Database for Multiple Sites?	23
Think About This...	23
Do This...	24
Update the Site Spreadsheet	24
Background Information	24
Chapter 6 How Should Sites Be Grouped?	25
Think About This...	25
Do This...	25
1. Create a List of Potential Groups	25
2. Update the Site Spreadsheet	25
Background Information	26
Chapter 7 How Should Configurations Be Grouped and Managed?	27
Think About This...	27
Do This...	27
1. Add Configuration Names and Groups to a Spreadsheet	27
2. Determine What Configuration Groups You Need	28
3. Decide Which Server Is the Configuration Server	28
Background Information	28
Configuration Names	28
Configuration Server.	29
Configuration Groups	31
Chapter 8 How Many Mongoose Intranets Do You Need?	33
Think About This...	33
Do This...	34
Review Your Physical Requirements	34
Update the Site Spreadsheet	34

Background Information	34
Intranets	34
Sending or Receiving Business Object Documents from Multiple Sites on an Intranet	36
TaskMan Monitoring of Sites	36
Chapter 9 How Do You Want to Use Web Rendering?	39
Think About This...	39
Do This...	40
Chapter 10 Which Data Needs To Be Shared Between Sites?	41
Think About This...	41
Do This...	41
1. Update the Flowchart	41
2. Create a ReplicationRules Spreadsheet	42
Background Information	42
Replication with Multiple Sites per Database	42
Replicating Data via BODs (ESB and ION)	42
Chapter 11 Should Some Data Be Shared Through Master Sites Instead of Replication?	43
Think About This...	43
Do This...	44
1. Update the Flowchart	44
2. Update the Site Spreadsheet	45
3. Create a List of the Tables or Categories Controlled by the Master Site	45
4. Update the ReplicationRules Spreadsheet.	45
Background Information	45
Intranets with Shared Tables and Master Sites	45
Shared _All Tables	45
Shared User Tables	46
Frequently Asked Questions About Shared Tables and Master Sites	46
Chapter 12 If Replicating Data, How Often?	51
Think About This...	51
Do This...	51
Update the ReplicationRules Spreadsheet.	51
Background Information	52
Replication Rules Form	52

Chapter 13 Are You Using the Event System?	.55
Think About This...	.55
Do This...	.55
Background Information	.55
Monitoring Events	.56
Chapter 14 How Many Licenses (and What Types) Do You Need?	.59
Think About This...	.59
Do This...	.60
Create a License Spreadsheet	.60
Background Information	.60
Chapter 15 Is Your Naming Scheme Logical?	.61
Think About This...	.61
Do This...	.62
Update Flowchart and Spreadsheets	.62
Chapter 16 How Easily Will Your Structure Incorporate Later Changes?	.63
Think About This...	.63
Do This...	.63
Revise Your Flowcharts and Spreadsheets	.63
Appendix A Replication Overview	.65
About Replication	.65
_all Tables	.66
Replication Categories and Rules	.68
How Categories Are Used with Rules	.68
Point to Point Replication	.68
Intranets	.69
Transport Protocol	.70
Master Sites and Shared Tables	.70
Multiple Sites in One Database	.70
Replicating _all Tables	.70
Replicating Base Tables	.71
Replicating Shared Tables	.71
Glossary	.73
Index	.75

About This Guide

Use this guide to plan for a multi-site installation of Infor Mongoose. When you have multiple sites, the installation and configuration of your system becomes much more complex. If you answer the questions in this guide before you begin installation, you will be prepared, and your installation and configuration should go more smoothly.

This guide has these sections:

- Planning Worksheet: Lists the questions, in order, that you should ask and answer when planning your system. Each step lists one or more tasks that must be completed while answering the question.
- Questions: Multiple chapters that:
 - Address each of the questions from the Planning Worksheet
 - List the planning tasks you must perform to answer the questions
 - Provide background information that helps you complete the planning tasks
- Glossary: Defines Mongoose and industry-specific terms used in this guide

Resulting Tools

After you complete the planning tasks, you should have these tools:

- A flowchart showing all of the sites and entities in your logical/financial hierarchy, and the shared data relationships between them
- A Site spreadsheet that lists, for each site, most of the information required for installation, such as site name, type, database names, site group, intranet name, and so on
- A ReplicationRules spreadsheet that lists each of the rules needed for replicating data between the sites
- An Events spreadsheet listing the configurations to be monitored by the Application Event System
- A Licensing spreadsheet that lists each license module (along with the number of users) needed for each site
- A Configurations spreadsheet - if you have multiple configurations per application database - that lists the configuration names, groups, and the related databases
- Lists of your planned site groups and master site tables

Use these tools, along with the *Multi-Site Implementation Guide* and the *Infor Mongoose Installation Guide*, to install and set up your system. Keep the information in the spreadsheets up to date, to use for reference as your system changes.

Planning Worksheet

Use this worksheet to determine the questions to ask and the tasks that need to be completed before you start installing your system. The questions are addressed in more detail, with information to help you answer them, in the step chapters listed below.

Step	Question	Tasks
1.	What is your corporate structure?	1. Draw a preliminary flowchart in pencil or using a computer modeling tool like Visio® - you will probably change the flowchart several times. 2. Create a Site spreadsheet containing a preliminary list of the site names and types.
2.	How many sites do you need? Determine the optimal number of sites for your system. Then determine some basic information about each site.	1. Update your flowchart with any changes to the site reporting structure. 2. Update your Site spreadsheet with any changes to the list of sites. For each site, specify the time zone, SQL server name or local node name, computer and domain name.
3.	Can you use the same Forms database for multiple sites? Do you need multiple Forms databases, and if so, which application databases should each one be linked to?	Update your Site spreadsheet to specify the Forms database used with each site.
4.	Can you use the same Objects database for multiple sites? Do you need multiple Objects databases, and if so, which application databases should each one be linked to?	Update your Site spreadsheet to specify the Objects database used with each site.

Step	Question	Tasks
5.	How should sites be grouped? Do you need multiple groups for different multi-site functions or users?	1. Create a list of potential Site Groups. 2. Update the Site spreadsheet to specify which Site Group the site should reside in initially.
6.	What configurations do you need? Do you need different sets of configurations to be available to different sets of users? Will you need multiple utility servers and thus a designated configuration server?	1. In the Site spreadsheet, add the configuration name and application database name for each site - or else create a Configuration spreadsheet. 2. Determine what configuration groups you need. 3. Decide which web/utility server is the Configuration Server.
7.	How many Mongoose intranets do you need? Review your physical requirements - web servers, utility servers, etc. Decide on the intranet names, and which sites will belong to each intranet.	Update the Site spreadsheet to indicate the name of the Mongoose intranet to which each site is connected.
8.	How do you want to use Web rendering?	Decide which utility server you want to use for the Web client, and determine the URL to access the Web client.
9.	Which data needs to be shared between sites? What categories of data need to be replicated? Which sites need to share that type of data? Does data need to be shared in both directions?	1. Update the flowchart with arrows indicating the categories of data being shared, and the directions the data is flowing. 2. Create a ReplicationRules spreadsheet listing each of the categories you think you will need, in each direction, from each site.
10.	Should some data be shared through a master site and SQL views rather than through replication? You may decide to set up master sites and shared views for some areas of your system.	1. Update the flowchart with boxes indicating master sites, if used. For data flowing to a master site from using sites, use a different type of arrow. 2. Update the Site spreadsheet to indicate any sites that are master sites. 3. Create a list of the tables that are controlled by the master site. 4. If some of the rules you thought you would need have been replaced by the use of a master site, remove the rules from the ReplicationRules spreadsheet.

Step	Question	Tasks
11.	If you will replicate data, how often? Should you use transactional or non-transactional replication?	Update the ReplicationRules spreadsheet to indicate which rules are transactional or non-transactional. If using non-transactional rules, specify the interval type and timing.
12.	Will you be using the event system?	Create an Events spreadsheet listing the configurations to monitor. Consider how event data might be replicated.
13.	How many licenses (and what types) do you need? Determine which users - and how many users - will need to work in multiple sites.	Create a License spreadsheet listing the sites and the license modules and number of licenses required for each site. If some sites use intranet licensing, note their intranet's master site in the spreadsheet.
14.	Is your naming scheme logical? Name sites, configurations, intranets, and site groups to indicate the logical structures and relationships.	Update the flowchart and spreadsheets with any name changes.
15.	How easily will your structure incorporate later changes? Think about how radical changes to your company hierarchy may affect the structure.	Revise your flowchart and spreadsheets if you decide to rearrange your hierarchical structure.

Caution: Planning and implementing a multi-site system is not a simple task. We recommend that you discuss the process and the results of your planning decisions with Infor Consulting Services prior to implementation.

Chapter 1: What Is Your Corporate Structure?

1

Think About This...

What hierarchical structure does your company require for:

- Sharing of administrative data
- Sharing of other types of application information

Do This...

Review the Background Information. Use that information to perform the tasks below.

1. Create a Flowchart Showing Your Hierarchy

Draw a flowchart showing your company's site hierarchy. Use a pencil, or use an application that lets you draw flowcharts on your computer, because you will probably change the flowchart several times. You will use the information in later chapters of this guide to help you refine your flowchart.

2. Create a Site Spreadsheet

Create a spreadsheet named **Site** that lists the following information for each potential site:

- **Site ID:** Up to 8 characters. Avoid using the following special characters in site names or IDs:
 - \ (back slash)
 - / (forward slash)
 - :
 - *
 - ? (question mark)

- " (double quote)
- < (left arrow)
- > (right arrow)
- | (vertical bar)
- (embedded space)
- **Site Name:** Up to 60 characters, including spaces. This is the long name for the site. It appears along with the Site ID on several Mongoose forms. (In some forms, the field displays only the first 25 characters or so.) For example, if your Site ID is ONT, your Site Name could be Ontario.
- **Site Description:** Up to 40 characters, including spaces. This text could describe both the location of the site and what it is used for. For example, "Illinois Site - Distribution."
- **Site Type:** Specify Site.
- **Application database name:** Use standard SQL database naming conventions. Application database names might include the site ID (for an environment with one site per database) or some other identifying characteristic. Generally you should end the name with the type of database; for example ONT_App.

You will add more columns for each site in later chapters. This information will be used when creating and configuring sites.

Background Information

Logically, a site is any place where work is done.

Sites are defined during Mongoose database server installation. Additional information about each site is set up on the Sites form.

This information includes the *intranet* used by the site and any linked sites. *Replication* rules may be set up to transfer data between sites, or *master sites* may be specified to maintain data for multiple sites. These features are discussed in the steps dealing with replication starting on page 41.

Site groups may also be set up to combine data for sites performing related functions. Sites in a group need to share the appropriate data through replication. This is discussed further in Chapter 6, "How Should Sites Be Grouped?"

You can add or remove sites later, as required by your situation.

Chapter 2: How Many Sites Do You Need?

2

Think About This...

Determine the optimal number of sites for your system. Then determine some basic information about each site.

Time Zones

You must specify a time zone for each site when the site is created. The time zone for the site is independent of the time zone for the server. Multiple sites running on one server can have different time zones. The time zone for each site should be appropriate for the users of that site.

When transactions are replicated between sites, the transactions are stored in `_all` tables and use the date and time the transaction occurred at the creating site.

SQL Server Name or Local Node Name

When you create application databases on the database server, you must specify the SQL Server Name where the database will reside. If using SQL Clustering, you will enter the SQL instance of the local node in the SQL Server Name field.

Do This...

Review the information in this chapter. Use that information to determine the optimal number of sites and then perform the tasks below.

Update the Flowchart

If you add or remove sites, or move them around in the hierarchy as a result of the information in this chapter, update your flowchart.

Update the Site Spreadsheet

Update this spreadsheet with any changes to the list of sites.

For each site, specify this information:

- The time zone in which the site operates
- The SQL Server name - or local node name, if using SQL Clustering
- The computer and domain name on which the SQL server resides
- You may also include a column for the SQL user under which the sites will be created; however, if you always use the "sa" user, you may not need this column.

Background Information

Physical Requirements

Generally, it is best to keep all application databases that will be sharing data through *transactional* replication on the same database server or at least in the same physical data center.

Chapter 3: Multi-Site per Database, or One Site per Database?

3

This chapter explains the differences between an environment with multiple sites all in one SQL database, versus an environment with each site in a different SQL database.

You can also set up an environment where you have multiple sites in one database, and another site in a separate database.

Think About This...

What are the advantages of one environment over the other?

Topic	Multiple Sites per Database	One Site per Database
Maintenance	Pro: Because you have only one database, backups, administration of indexes, etc. are easier. Con: Backups and restores take longer to complete for all sites participating, and will take progressively longer over time, exponential to the number of sites. Increased time to run index rebuilds, analysis, statistics, etc. Con: Only one recovery model can be selected.	Pro: Backups and restores can be done on the databases (sites) during the various sites' maintenance windows, based on each site's operations low utilization or down times. Con: Administration of multiple databases for indexes, statistics and database maintenance is more complex and potentially more time-consuming. Con: Data recovery, high availability, and disaster recovery could require coordination between multiple database (site) failover points for all of the application databases.

Topic	Multiple Sites per Database	One Site per Database
Upgrades and issue fixes	Pro: Because you have only one database, upgrades are easier. Pro: You have one copy of the stored procedures and functions that is used by all of the sites in the database. So when you upgrade or install bug fixes, you only have to update one database. Con: When you apply single fixes that involve core database logic (method calls) such as stored procedures, etc., the fix impacts all sites.	Pro: Single sites can be upgraded to new versions of Mongoose, rather than having to simultaneously train and convert all sites at once. (However, this gets more complicated if the site is replicating data to other sites.) Con: You have multiple copies of the stored procedures and functions in each site SQL database that is used by the specific site. So when you upgrade or install issue fixes, you must update all site databases.
Processing speed	Pro: If all your sites are in one database, tasks that require performance of queries, stored procedures or functions at another site may display results more quickly at the originating site, because they can take advantage of shared memory. Con: Writes to the log file of a single instance database with multiple sites could create a bottleneck at the log file. Log file maintenance is more important in this scenario.	Pro: Each site can be separately administered and tuned to best perform for the usage of that site (memory allocation, backup schemes, number of active sessions, etc.) Con: Collecting and querying data from multiple databases is less efficient. You need multiple execution plans and connection sets.
Data integrity	Pro: Keeping data in sync between sites is easier, especially after a restore	Con: You could lose some multi-site data integrity when you back up and restore individual databases in various states that are participating in a multi-site environment.
System down	Con: If the database goes offline, all sites are down. If you have to do a system restore or backup, it affects all sites in the database.	Pro: When you take a site database offline, there is little or no impact on other site databases.

Topic	Multiple Sites per Database	One Site per Database
Scalability	Con: The database server that is processing data for the single database with multiple sites will need to scale up as the database grows in size and usage. The ability to distribute site load across multiple database server infrastructure/instances would not be effective.	Pro: If sites reside in their own databases, you can distribute of those databases across multiple database server infrastructure/instances.
Objects database	Con: Only one objects database can be used with all sites in the application database	Pro: Sites can use different objects databases.
Licensing	Con: A shared license is required.	Pro: You can license each site separately, or use a shared license.
Portability	Con: Use the “Add new site to an Existing Database” option to copy the site to a new database, but no option to remove one site from an existing multi-site database.	Pro: Sites are physically separate in their own database, so portability is less of an issue. A site and its contents can be moved or removed if necessary.
SQL requirements	Con: The Enterprise version of SQL Server is required.	Pro: Either the Standard or Enterprise version can be used, except when other factors such as required memory, etc. indicate that the Enterprise Edition is needed.
Other considerations	Pro: UETs and events are shared between sites. (This can also be a “con.”) Con: You cannot isolate site-specific procedure changes, either programmatically or from a SQL security perspective. A change made to a stored procedure is visible to all sites in the same application database. Coding to exclude sites could be implemented in custom stored procedures.	Pro: Sites are logically separate, each with a complete set of data and programs. Pro: Performance tuning and troubleshooting have less impact on the other sites running on the infrastructure.

Topic	Multiple Sites per Database	One Site per Database
Other considerations (continued)	Con: If you turn on performance tuning or troubleshooting (for example, Profiler, execution plan estimation, etc.), it impacts all sites. For tables that contain large binary fields, rebuilding an index locks the table for all sites.	

Do This...

Decide Whether You Need Multiple Sites in One Database

Use the background information to determine whether your company needs a multi-site system.

Background Information

Switching from One Option to the Other

You can use the Configuration Wizard to **copy** a site from one database into another, which lets you merge sites into a single database or split sites into individual databases.

The copied site remains in the source database. If any other remaining sites are still valid in the source database, they must be moved to a new database, because there is currently no way to **remove** a site from a database.

Differences Between Single Database vs. Multiple Databases

_All Tables vs. Base Tables

If sites are in different databases, the _all tables in each site database contain data specific to other sites on the current site's intranet. The base tables in each site database contain only data specific to the current site. The _all views point to the base tables.

If all sites are in one database, the `_all` tables still exist but do not contain any data. They are simply views pointing to the base tables. The base tables contain data for all sites in the database, including the current site, filtered by the `site_ref` column value. However, if at least one site listed in the sites table for an application database is in a separate application database, then the `_all` tables are populated, and the `_all` views point to the `_all` tables.

Event Handlers

Application Event System events and event handlers (including the **Active** option) are defined for a database, not for an individual site. However, you can use the **Applies to Sites** field to specify the sites for which an event handler is active.

For more information about event handlers, see the *Guide to the Application Event System*.

Speed of Replication

Replication between sites is faster if the sites are on the same SQL server. However, replication between sites within the same database is not any faster than for sites in different databases on the same server.

Multi-Site Tables and Shared Tables

Multi-site tables include a `site_ref` column that indicates the site to which a row of data belongs. Most application tables are multi-site. Several Mongoose tables are also multi-site.

Most Mongoose tables, and a few application tables, are considered to be “shared” by all sites in a database.

Data Visibility

If you have two sites in the same database that are not replicating data between each other, and are not sharing `_all` table data, a SQL user with direct database access to one of the sites can still view base table data for both sites. If that is not desirable, then the two sites should not be placed in the same database.

Process Defaults

Settings on the Process Defaults form apply to all sites in a database.

Modified Reports

SSRS reports use Background Task Definition executable file names that are the same across a database. In a multi-site, single database system, if you need to modify report tasks so that they produce different outputs for different sites, you can set up SSRS subfolders that match the sites' configuration names. The system looks first in the SSRS subfolder that matches the configuration name for a report to run. If it doesn't find it, the system defaults to the SyteLineReports folder.

For form-based reports or Dataview reports, you can use the scope level to distinguish between sites.

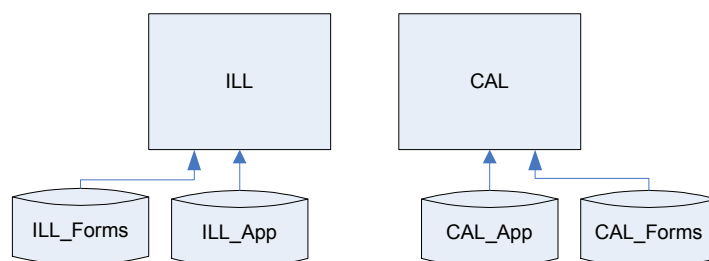
Chapter 4: Can You Use the Same Forms Database for Multiple Sites?

4

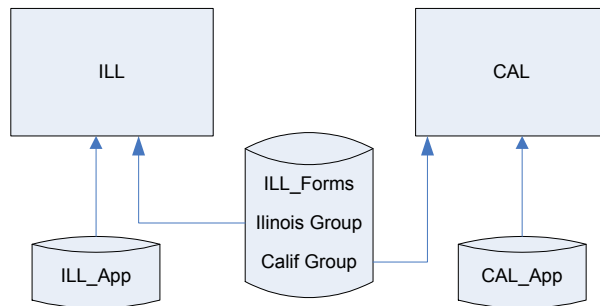
Do you need multiple forms databases? If so, which sites should each forms database be linked to?

Think About This...

- The Configuration Wizard has an All-in-one database option that allows you to keep all of your forms, objects, and templates metadata in the same database as your application data. If you select that option, the references in this guide to the “Forms database” should be changed to the “Forms metadata tables in the application database.” The concepts are still the same. However, be aware that, if you use the all-in-one option for a database, you can only share forms, objects and templates metadata for sites that are in that application database.
- If you have multiple sites, and they will be looking at the same version of the Mongoose application and the WinStudio framework, it might make sense to link them all to a single forms database. This allows you to create form customizations that all the users at those sites would see.
- You might want to use different forms databases for different communities of users. For example, some users may need to see a set of custom forms or custom fields, and other users would not require them. For example:



- However, using a single forms database, you can target customizations by group. For example:



- Having one forms database for multiple sites can save space on your database server machine. And since forms database "traffic" is mostly read-only access, having many users/sites sharing a forms database usually does not present performance problems.
- You may want to put your forms database on a separate server from your application database to enhance application performance. If you do this, consider the speed of the network between the forms database server and the utility server. For example, the forms database server probably should not be in a different data center from the utility server.

Do This...

Review the background information on the following pages. Use that information to perform the tasks below.

Update the Site Spreadsheet

Update your Site spreadsheet to specify the forms database to be used with each site.

Note: The Mongoose configuration wizard will assume that the forms database name is the same as the application database name (for example, Crp_App would have a forms database called Crp_Forms). You can select a different forms database by changing the default value.

Background Information

The forms database stores specifications for form components, validation procedures, scripts, variables, and other objects that make up the Mongoose forms (in essence, the user interface). Data specified in design mode is stored in the forms database, and the framework interprets the data at run time to create the form.

The forms database includes a set of "strings" tables, which contain translations of the text strings used in form titles, field labels, buttons, and so on. The default strings table, for English, is named

Strings. The other strings tables have names like JapaneseStrings, FrenchStrings, and so on. The strings table used for each language ID/culture is determined by the settings in the application's Language IDs form.

If you are customizing Mongoose, we recommend maintaining a separate, work-in-progress forms database that is accessed by developers using WinStudio design mode. This work-in-progress database stores product changes until they are tested and ready to be copied to the deployed forms database accessed by end users.

The value you specify in the **Forms Database Name** field on the Sites or Sites/Entities form points to the forms database that contains the strings tables you want a specific site to use. The value in this field should match the **Runtime Forms Database Name** that is specified in the Configuration Manager utility for the primary configuration used to log into this site.

More information about the use of the forms database is available in the FormControl and FormSync utilities' online help.

Chapter 5: Can You Use the Same Objects Database for Multiple Sites?

5

Do you need multiple objects databases, where the IDO definitions reside? If so, which sites should each objects database be linked to?

Think About This...

- The Configuration Wizard has an All-in-one database option that allows you to keep all of your forms, objects, and templates metadata in the same database as your application data. If you select that option, the references in this guide to the “Objects database” should be changed to the “Objects metadata tables in the application database.” The concepts are still the same. However, be aware that, if you use the all-in-one option for a database, you can only share forms, objects and templates metadata for sites that are in that application database.
- IDO editor forms in Infor Mongoose are used to create and maintain the IDOs metadata, which is then stored in the objects database and used by the IDOs at run time.
More information about the use of the IDO editor forms is available in the online help. These forms are accessible only with a separate "development" license.
During configuration, the objects database is specified for a site. The application database contains views into the objects database, so the runtime environment does not directly access the objects database.
- Reasons to share a single instance of the objects database for multiple sites include the following:
 - Any IDO patches, add-ons, or customizations are shared.
 - The application uses less disk space.
- You might **not** want to share an instance of the objects database across sites if the sites are in different data centers, and you do not want to pay the performance cost of having to access the objects database across a slow internet at run time.

Note: If you have multiple sites in a single application database, all of the sites **must** use the same objects database.

Do This...

Review the information on the previous page. Use that information to perform the task below.

Update the Site Spreadsheet

Update your Site spreadsheet to specify the objects database to be used with each site.

Note: The Mongoose configuration wizard will assume that the objects database name is the same as the application database name (for example, Crp_App would have an objects database called Crp_Objects). You can select a different objects database by changing the default value.

Background Information

For more information about IDO maintenance forms and the objects database, see the online help.

Chapter 6: How Should Sites Be Grouped?

6

Do you need multiple groupings of sites for different multi-site users or functions?

Think About This...

- Site groups may be set up to combine data for sites performing related functions. Sites in a group must share the appropriate data through replication or shared tables.
- You can define groups containing different combinations of sites. A site can be included in more than one site group.

Do This...

Perform the tasks below.

1. Create a List of Potential Groups

Make a list of the site groups your system might use. Later, you can assign these groups to sites through the application's Site Groups form.

2. Update the Site Spreadsheet

Update your Site spreadsheet to specify which site group each site should *initially* belong to. An initial group is required when you configure the site.

Background Information

Review the background information about multi-site on page 10.

.

Chapter 7: How Should Configurations Be Grouped and Managed?

7

Do you need different sets of configurations to be available to different sets of users? Will you need multiple utility servers and thus a designated configuration server?

Think About This...

- Each site has at least one configuration defined for it through the Configuration Manager. Multiple configurations may be defined for one site.
- Configurations may be grouped to prevent users from accessing certain sites.
- You can designate one web/utility server as the Config Server.

Do This...

Perform the tasks below.

1. Add Configuration Names and Groups to a Spreadsheet

On the Sites/Entities form, specify a default Configuration Name for the site. You must create a configuration with this name, which will be mapped to this site record.

Alternatively, if you leave this field blank, you must create a configuration whose name exactly matches the local site name, including the capitalization. For example, if your local site name is oh, you must create a configuration named oh. You can use the Copy feature of the Configuration Manager utility to create this configuration

You may need multiple configurations, with different names, for one site.

If you plan to have only one client configuration for each site, then add the configuration name to the **Site** spreadsheet.

However, if you will have multiple configurations for the same site, create a new Configurations spreadsheet that contains the following information:

- Configuration Name
- Configuration Group(s) to which it belongs
- Site
- Application Database
- Forms Database
- Objects Database
- Utility Server (or Configuration Server if pooling servers)

2. Determine What Configuration Groups You Need

You may want to restrict the list of configuration names that users see to a subset of the defined configurations. See the background information for details.

3. Decide Which Server Is the Configuration Server

In a large multi-site system, you may have multiple web/utility servers. In this case, a designated Configuration Server is required. Read the background information to see how the Configuration Server is used.

Background Information

Configuration Names

You will be asked to create at least one *client configuration name* for each site. A site can have multiple configurations associated with it.

The configuration name is what the user sees and selects at login - so it should be descriptive and easily distinguished from other sites' configurations. For example, you might have two configurations for the ONT site called ONT_SALES and ONT_WHSE, using different forms databases but the same application database.

The maximum length for a configuration name is 50 characters and can include spaces. However, only the first 25 or so characters are displayed in the selection drop-down list on the Mongoose login

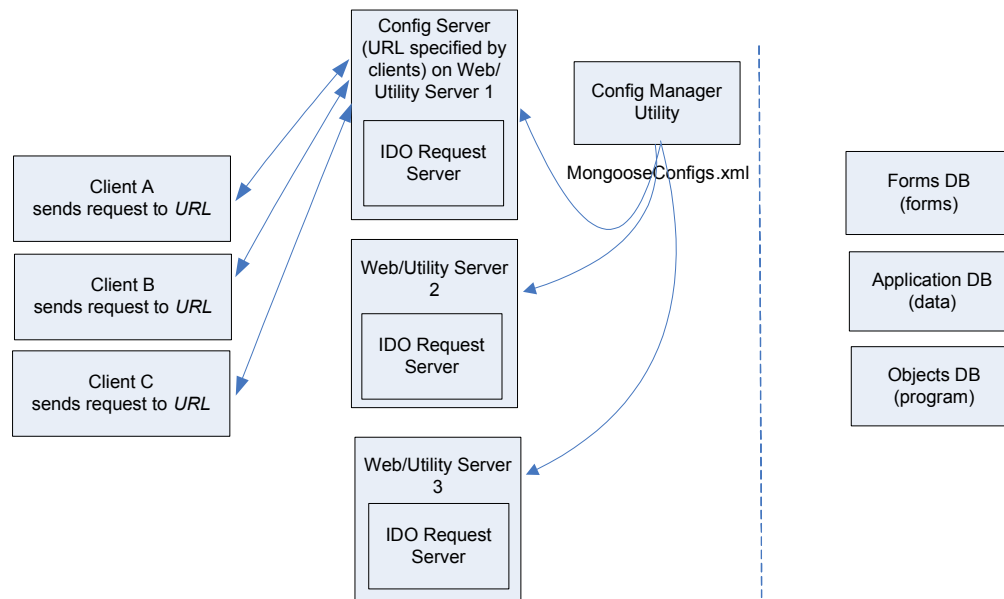
screen. To allow users to see the entire configuration name, we recommend that you keep the names shorter than 25 characters.

Differentiation of configuration names should be obvious in the first five characters or so. This allows you to quickly distinguish between configurations when you have several Mongoose sessions open at once.

Configuration Server

In a large multi-site system, you may have multiple web/utility servers. In this case, a designated Configuration Server is required, because clients have one initial URL, pointing to a single machine. Only after users log in will the client session be directed to a member of the utility/web server pool.

GetConfigurations Request



When a user logs into a WinStudio (Mongoose) session on a client, the system looks for a URL in either of these places:

- In the application shortcut's properties, a **-s** parameter is followed by the URL, or
- In click-once client deployments, the URL is found in the **ConfigServerURLs.xml** file on the client.

The client sends a "GetConfigurations" request to this URL, which points to the **ConfigServer.aspx** page on a utility/web server that the administrator has decided to use as the "Configuration Server."

When this request comes in, the ConfigServer.aspx reads the **MongooseConfigs.xml** file, which also resides on the utility server. This XML (created with the **Configuration Manager** utility) contains information about the following:

- Configurations - named combinations of specific application, forms, and objects databases.

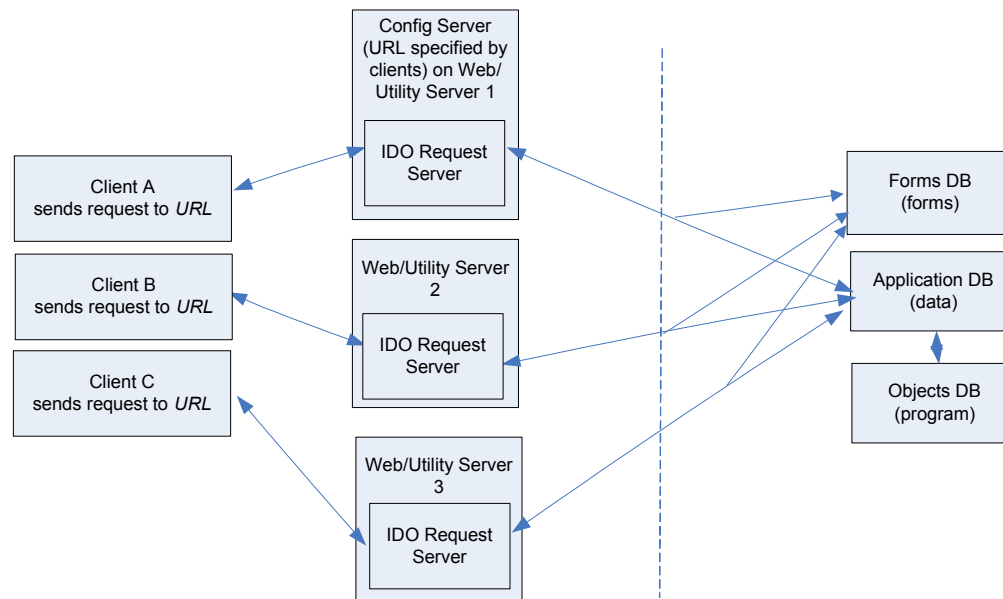
- Applications - specifications of themes, etc.
- Web Servers - a list of available utility/web servers. Do not include DMZ servers in this list.
- Configuration Groups - defined subsets of the full list of configurations available on a server.

Each time a GetConfigurations request comes in, ConfigServer.aspx picks a utility/web server from the list, in round-robin order; that is, it selects the next one in the Web Servers list in MongooseConfigs.xml.

It returns to the client the URL of the **IDO Request Service** on the selected utility/web server. This service can connect to all available configurations listed in its copy of MongooseConfigs.xml. The list is limited by the configuration groups the client can access.

IDO Requests

For the remainder of that session, the client will send all requests to the specified IDO Request Service URL, which runs the request and returns the response back to the client.



Designating a Configuration Server

Run the **Configuration Manager** utility only on the web/utility server that you want to designate as the Configuration Server. This creates the **MongooseConfigs.xml** file. Then copy that XML file to the same place on all other web/utility servers. Whenever you need to update configurations or your list of web servers, run Configuration Manager on the Configuration Server machine and redistribute the MongooseConfigs.xml file to the other servers.

On all clients, specify the same **URL**, which points to the web/utility server designated as the Configuration Server. For example:

`http://server1/IDORequestService/ConfigServer.aspx.`

Configuration Groups

When users start Infor Mongoose, they are presented with a list of configuration names to select from. By default, this is a list of all the configurations that are defined on the Configuration Server. However, you may want to restrict the list of configuration names that users see to a subset of the defined configurations. For example, you could hide test, pilot or other configurations from users who should not access them.

You set up configuration groups, and specify the configurations that are members of each group, through the Configuration Manager utility. Then, when setting up clients, you can specify a group name as part of the connection information:

- If a client is using WinStudio on a utility server with no -s command line option, you can specify the configuration group using the -g<config-group> command line option. For example:
WinStudio.exe -gAcme
- If a client is connecting to the Configuration Server using http, the configuration group can be specified as a query parameter in the URL. For example:
http://slutil/IDORequestService/ConfigServer.aspx?ConfigGroup=Acme
- If you are using XML to connect, you can specify the configuration group name as part of the GetConfigurations IDO request. For example:

```
<IDORequest>  
<RequestHeader Type="GetConfigurations" ConfigGroup="Acme"/>  
</IDORequest>
```

Chapter 8: How Many Mongoose Intranets Do You Need?

8

The Intranet feature provides a way to logically show your LAN/database site setup from within Infor Mongoose. Determine the number of intranets needed, decide on the intranet names, and decide which sites will belong to each intranet.

In most cases, you will need only one intranet. The exceptions are listed in this section.

Think About This...

- How are the sites physically connected? Generally all the sites on an Mongoose logical intranet should be on the same physical LAN/intranet, especially if you plan to use *transactional replication*. Although it may be possible to run transactional replication between sites on different Mongoose intranets, it is not recommended if the sites are connected by a slow or unreliable network connection.
- Will you have different versions of Mongoose running at different sites? (This is not typical.) If so, they must be on different intranets.
- There can be only one *master site* per intranet. Master sites are explained on page 43.
- Are you planning to use *intranet licensing*, where you enter and maintain licensing information at one master site for all sites on the intranet? This concept is explained in the *Licensing* guide.
- Will you have an *external* intranet, which allows integration, via replication, to applications other than those based on the Mongoose framework? Replication performed to a site on an external intranet must be non-transactional.
- Clients may be deployed to key customers or suppliers over the internet, allowing them access to specific system features. In this case, you probably should configure your web server components (and the URL for that server on the Intranets form) to use https instead of http, so that the data going over the internet is encrypted.
- Will you be pooling multiple utility servers through a *Configuration Server* (page 29)? If so, we generally recommend that a Configuration Server and all its pooled utility servers, and all the clients using that Configuration Server, be on the same intranet in the same datacenter, for high-speed, high-reliability connectivity. However, it is possible to have two intranets in the same datacenter (for reasons of logical grouping) and use a single set of utility servers to make the configurations available to end users for sites on both intranets.

- Are you planning to have multiple sites send and receive business object documents (BODs)?
- How will TaskMan be configured to monitor sites on an intranet, especially in a setup with multiple utility servers (page 36)?
- There can be only one replication queue listener defined per utility server. For non-transactional replication, each intranet requires a replication queue listener. Therefore, if you have multiple intranets in your data center, you must have *one utility server for each intranet* where the replication services are configured to monitor that intranet for non-transactional replication.
- If you have multiple sites in a single application database, and if the sites are sharing `_all` or user tables (through the Intranet Shared Tables or Intranet Shared User Tables utilities), then all sites in that database must be on the same intranet.

Do This...

Review Your Physical Requirements

Determine your system's physical requirements.

Physical requirements include the number of utility servers, web servers, clients, and so on. If you are using multiple utility or database servers, you may want to update your hierarchy flowchart to indicate which databases and configurations will be used on which servers.

Update the Site Spreadsheet

Update your Site spreadsheet to indicate the name of the Mongoose intranet to which each site is connected. Try to keep this spreadsheet up to date.

Background Information

Intranets

Mongoose intranets represent logical groupings of sites in your enterprise. An intranet may represent a grouping of Mongoose sites that exist together on a high-speed network, and for which you want common administration. (Common administration can include replication or creation of master sites.)

An intranet may also represent an external non-Mongoose system, allowing you to set up replication rules between Mongoose sites and external systems.

Use the Intranets form in the application to define information about your logical intranets. Then use the Sites or Sites/Entities form to assign your sites to an intranet.

For example, if you have two sites in one datacenter and two sites on a different datacenter, define two intranets, one for each datacenter, and put the sites from each datacenter on those intranets.

External Intranet

Create an *external* intranet if you need to set up replication between sites on one of your Mongoose intranets and an external system or an application.

For such external systems or applications, the only attributes that matter will be the intranet name (so you can define "sites" on the Sites or Sites/Entities form that represent instances of external systems), the transport method (see below), and the URL. When you have defined sites on such an external intranet, you can then create replication rules (see page 51). The replication system will dispatch XML documents to the URL you have specified on the intranet.

An Mongoose site can communicate with an external site when it is on a different intranet, as shown in the following example:

MyCORP (Internal) Intranet:

SiteA

SiteB (master site)

SiteC

InforBUS Intranet (BOD-enabled applications):

InforBUS site

Each of these external sites must be on its own intranet because each application uses a different URL address.

Mongoose SiteB could perform transactional replication with all the internal sites, and could perform non-transactional replication to all the external sites.

Transport Method

Most sites on external intranets use HTTP transport protocol to communicate with Mongoose sites. However, Infor BOD-enabled applications use the ESB transport protocol.

Sending or Receiving Business Object Documents from Multiple Sites on an Intranet

Inbound and Outbound BODs

You can specify a default configuration name for a site in the Sites or Sites/Entities form. Alternatively, you can create a configuration whose name exactly matches the local site name.

On the Mongoose utility server, you specify configuration information on the **Replication** tab of the Service Configuration Manager utility:

- In the **Replicator/Inbound Bus Configurations** field, specify all of the configurations that can receive inbound or outbound messages from other sites on the intranet. The Replication Document Outbox form at each site collects outbound BODs for that site. The Replication Document Inbox form at each site collects inbound BODs for that site.
- In the **ReplQListener Queue Servers** field, set up a queue server name for every intranet in your system.

Note: If you are upgrading from a previous version and you already have replication set up through a "bootstrap" site, it will still work as it did in the previous version. The **Replicator/Inbound Bus Configurations** field contains one site set as the Master Site, and the **ReplQListener Queue Servers** field has the **Default** field selected.

The named queue is the same for the specification in the Service Configuration Manager as it is in the **Queue Server Name** field on the Intranets form. For example, if you want utilityserver3\private\$\MyQueues to be looked at by ReplQListener, you would use that queue in the Service Configuration Manager as well.

Purging BOD information

Information from every inbound BOD, including sequencing information, is collected in a table in the application. This data constantly builds up and, over time, could cause performance issues. For this reason, system administrators should periodically use the Purge BOD Information form to purge information from that table. You can set up a background task to perform this function automatically.

You can also use the Replication Document Inbox/Outbox Purge Utility to clear out older BODs from the Replication Document Inbox and Outbox.

TaskMan Monitoring of Sites

The Infor Framework TaskMan service must be set up to monitor each site on which users will be executing reports, stored procedures, IDO methods, executables, or other background tasks.

During installation, a shared folder called Mongoose is set up on the utility server. When you enter information on the Intranets form, you must specify the path to this shared folder as the TaskMan Path. That path, which is used for all sites on the intranet, is usually set to `\\utility_server\Mongoose`, where *utility_server* is the server containing the shared folder.

Using the Service Configuration Manager on the utility server, you specify which configurations will be monitored by TaskMan on that server.

If You Have Multiple Utility Servers

You may have more than one utility server with identical configuration settings, pointing at the same application databases. You can configure the TaskMan service on each of the utility servers so that TaskMan monitors the same configurations on each server.

One utility server could be used as a "report server" and would contain the shared folder Mongoose. That server would be used for processing all reports, background tasks, etc. for the entire system.

You can have more than one TaskMan service instance monitoring the same application database (ActiveBGTasks_mst table). Whenever task requests come in from a site, the appropriate utility server's TaskMan service would pick them up. The TaskMan services on all the utility servers would access the shared UNC path (specified in the TaskMan Path on the Intranets form).

When initiating TaskMan, the system calls a stored procedure to retrieve the TaskMan Path value; then all report task requests are based on that path.

As long as each TaskMan service startup account has read/write privileges for the report server shared folder (`\\RptServer\Mongoose`), and the TaskMan Path for each site's intranet is pointing at the appropriate shared UNC path, the system can handle the following tasks from any site:

- Look for requested report files on the report server
- Direct Microsoft SQL Server Reporting Services (SSRS) to generate output files in a specified output folder
- Create error/log files on the report server

Chapter 9: How Do You Want to Use Web Rendering?

9

The Mongoose framework allows you to display forms in Web browsers. Two Mongoose clients are available for use with browsers:

- The Mongoose *smart client via browser* is available for use in Internet Explorer. It runs in a Windows environment and requires a small footprint on the client machine.
- The Mongoose *Web client* can be used with most popular internet browsers. It has a zero footprint.

This chapter describes the use of the Web client, not the smart client via browser.

Think About This...

- Do you want your users to be able to view forms through the Web client? Reasons your users may want the web client as opposed to the smart client include access on computers that require a zero client footprint (no file downloads), non-Windows computers, mobile devices, or use inside an Infor Ming.le web part.
- If you do want to include the Web client, do you want it to be run from a utility server that is also performing other work (servicing the smart client, running TaskMan, etc.) or do you want to set up a dedicated utility server for the web rendering engine?

Although firm guidelines are not defined, if you expect a very high volume of your users to use the Web client, then you might want to dedicate a utility server for this purpose.

- Use the DMZ Server installation option in deployments where the IDO Request Service and WSWebClient web applications are installed on a machine without the full utility server components. You can choose this option if you do not want to expose your utility server directly to the internet and do not want to place a load balancer or other hardware between the utility server and the internet.

Do This...

If you want to use the Web client, identify the utility server(s) that will run the client. Include the Web client option when you install Mongoose on that utility server.

After installation, you must provide the Web client login URL, which includes the server name, to your Web users.

For more information about the installation and setup of the Web client, see the *Infor Mongoose Installation Guide*.

Chapter 10: Which Data Needs To Be Shared Between Sites?

10

STOP: Read Appendix A, “Replication Overview,” to get a general understanding about how replication works before you continue. Without that understanding, the replication planning steps probably will not make sense.

Think About This...

You can place the data that needs to be shared in various replication categories. These categories must include all the necessary database tables, methods/stored procedures, and/or XML documents that are required to replicate a certain type of data between sites.

When you set up a multi-site system, you usually start by replicating categories between the sites where you think the information is required.

After your system is running, you might need to revise the rules, and possibly customize the categories, to achieve optimal performance based on your specific system’s needs.

List the categories of data that need to be shared through replication, and the sites that need to share that type of data.

Does the data need to be shared in both directions?

Do This...

1. Update the Flowchart

Update your flowchart with arrows indicating the categories of data being shared between the sites, and the directions the data is flowing.

2. Create a ReplicationRules Spreadsheet

Create a ReplicationRules spreadsheet listing each of the categories you think you will need, in each direction, from each site.

Do not specify whether the rules are transactional or non-transactional yet.

Background Information

Replication with Multiple Sites per Database

When you have multiple sites per database, tables are shared by the sites in that database. However, data is filtered by site. if you want a site to be aware of another site's specific data, you still must set up replication rules between the sites.

Replicating Data via BODs (ESB and ION)

This application provides an XML-based interface that can be used to transmit data to and from other Infor BOD-enabled applications. This interface must be set up as described in the appropriate application's integration guide.

Rules containing the ION and ESB replication categories should be used only where the "To" Site is a site that represents BOD integration.

Chapter 11: Should Some Data Be Shared Through Master Sites Instead of Replication?

11

STOP: Read Appendix A, “Replication Overview,” to get a general understanding about how replication works before you continue. Without that understanding, the replication planning steps probably will not make sense.

Think About This...

You may decide to set up master sites and shared views into tables at the master site, instead of replicating tables, for some areas of your system.

If all your sites are in a single database, there is no need to use a master site for sharing tables. You are effectively sharing all database tables already. However, if you want to use Intranet Licensing, you must specify a master site.

If you have one or more sites in one database, and additional sites in the same intranet but not in the same database, you can use a master site and the Intranet Shared Tables feature to allow sharing between the sites in one database and the sites in other databases.

Advantages of Using a Master Site

If you use a master site for certain functions, you can:

- Establish central global tables, many of which allow filtering by site
- Require less storage space – fewer records are replicated across the enterprise
- Use less background processing
- Simplify replication troubleshooting and querying of _all tables
- Provide a lighter load on database servers, thus better speed for end users.
- Centralize maintenance and validation of licenses if you are also using intranet licensing (see “Intranet Licensing” in the *Licensing* guide.)
- Centralize maintenance of user and group information.

Physical Requirements

- There can be only one master site for an Mongoose logical intranet.
- Live links must be set up (on the Sites or Sites/Entities form) between the master site and the other sites in the intranet.
- Do not share `_all` or user tables across multiple SQL Server instances, because it may severely impact performance. We recommend that a master site and its subordinate sites reside on the same server in a linked SQL environment. If you have multiple SQL Server instances, you can set up one intranet per SQL Server instance name and use regular replication between the two master sites on those two intranets.
- If a master site exists and has a shared table, all sites on that Mongoose intranet must share that table through views; you cannot have some sites sharing the table and some sites not sharing the table.
- If you have multiple sites in different application databases on one SQL server, then you can:
 - Set up transactional replication between the sites, or
 - Set up a master site that shares its tables with other sites on its intranet, or
 - Set up a combined system, where the master site holds some shared `_all` tables, but some other `_all` tables are replicated between the sites.
- If you have multiple sites on separate servers or on separate intranets, non-transactional replication is probably a better choice.
- If multiple sites exist in one database, then it is not possible to share tables in one site without sharing tables in all sites in that database.
- Consider the load on the database used for the master site and the number of users/sites that will need to access the master site. In large multi-site systems, you may want to designate a site to be used only for that purpose, for better system performance.

Do This...

If you decide to use a master site, perform the following steps. Otherwise, skip to the next chapter.

1. Update the Flowchart

Update your flowchart with boxes for any master sites. These boxes should indicate somehow - through shading or color, perhaps - that they are master sites.

Indicate shared data that resides at the master site - either by replication category or by table name. Use a different type of arrow, perhaps thicker or a different color.

Remember that all of the sites in the same application database must share tables in the same way. You cannot share a table in one site in database A without sharing the table for all sites in database A.

2. Update the Site Spreadsheet

Update the Site spreadsheet to indicate any sites that are master sites. You could do this either by adding a Master Site column or by adding "...Master Site" to the description.

3. Create a List of the Tables or Categories Controlled by the Master Site

For future reference when setting up the Intranet Shared Tables form, create a list of the shared tables (or the replication categories) controlled by each master site.

4. Update the ReplicationRules Spreadsheet

All sites on the master site's intranet must be replicating a category that includes the intranet table, in order to recognize the master site.

Background Information

Intranets with Shared Tables and Master Sites

On the Intranets form, you can specify a master site for an intranet. Master sites are Mongoose sites that control some data for all other sites on an intranet. For example, master sites control any shared `_all` tables or shared user tables.

An intranet defined as "External" (containing non-Mongoose sites) cannot have a master site defined for it.

You cannot delete an intranet that contains shared tables.

Shared `_All` Tables

Your application may include shared "`_all`" tables. See Appendix A, "Replication Overview," for an explanation of how these tables are used.

At the master site, use the Intranet Shared Tables form to choose the `_all` tables that you want to share — that is, the tables that should reside only at the master site for this intranet. Or you can select

a replication category, and the system automatically selects the `_all` tables that are included in that category.

The selected tables are removed from all sites on the intranet except the master site. SQL views to the table on the master site are created at each using (a.k.a "slave") site. The other sites can add, update, and delete records through the views.

You can also unshare a shared table from this form. The views at the using sites are dropped, and the `_all` table is rebuilt at each of the using sites on the intranet.

Shared User Tables

At the master site, use the Intranet Shared User Tables form to share the user maintenance tables so they reside only at the master site for this intranet. This feature requires Intranet Licensing to be implemented on all participating sites.

The tables are removed from all sites on the intranet except the master site. SQL views to the table on the master site are created at each using (a.k.a "slave") site. The other sites can add, update, and delete records through the views.

You can also unshare shared tables from this form. The views at the using sites are dropped, and the user tables are rebuilt at each of the using sites on the intranet.

Frequently Asked Questions About Shared Tables and Master Sites

Q: When would I want to set up a master site?

A: We recommend that you set up a master site even if you do not plan to share any tables at this time. (The only exception might be if you have only one or two sites to maintain.)

Master sites currently allow you to centralize maintenance of licenses and users.

Q: Why would I want to set up shared `_all` tables?

A: If your system is replicating a lot of data among many sites, processing time is reduced when the `_all` tables exist only at the master site.

Q: Why would I want to set up shared user tables?

A: It allows one administrator to maintain one set of users and user permissions in the master site that are applicable to all of the using sites on the intranet. The information includes:

- User information
- Groups information, including user-group assignments, user-group authorizations, and user-license module assignments.

You also have the option to exclude the `AccountAuthorizations_mst`, `UserGroupMap_mst`, and/or `user_local_mst` tables when you share user tables. This allows administrators to maintain users and most user settings in the master site, but apply different permissions for those users in each

of the sites. Or you can maintain authorization groups globally (so they are the same for all sites), but maintain the user groups locally, so users can be assigned to different permission groups at different sites within an intranet.

Q: Can I set up some tables to reside only at the master site, and other tables to be replicated at all sites?

A: Yes, for `_all` tables. No, for user tables.

Q: Which tables does it make most sense to maintain at a master site rather than replicating?

A: This depends on your data, and the parts of the system you use the most.

Q: Which tables can *only* be maintained from a master site?

A: If you are using intranet licensing, the license tables can only be maintained from the master site.

Other than that, you can maintain data at any site (slave or master site).

Q: Can I have a master site for one intranet and not for another one? If so, why would I want to do that?

A: Yes. The other intranet may not have large `_all` tables, or performance is good. Then for that intranet, there might not be a need for shared `_all` tables. Also note that if an Intranet is flagged as External, then it could not have a master site.

Note: If an intranet is set up to share tables, then only sites for that intranet can be placed in the same application database.

Q: Can I set up a master site any time, or just when setting up Mongoose initially?

A: At any time.

Q: Can I change the master site after it is set up?

A: Yes. Some of the `_all` or user tables at the slave sites have been dropped and would have to be rebuilt, and the data that resides only in the master site's tables would have to be reloaded into the rebuilt tables. You can do this in the Intranet Shared Tables or Intranet Shared User Tables forms.

If you remove a master site, intranet licensing will no longer be available, since this feature depends on having a master site.

Q: Can I have a master site on one intranet and no master site on another intranet, and still perform replication between sites on both intranets?

A: Yes.

Q: Why would I use a master site if all of my sites are in a single database?

A: The Multi-Site Customers, Multi-Site Items, and Multi-Site Vendors forms can only be used from a master site.

Since all sites are in the same database, most user tables are shared across all sites even without a master site and without setting up Intranet Shared User Tables. However, you can share these user tables only if a master site is defined, through the Intranet Shared User Tables form:

AccountAuthorizations_mst, UserGroupMap_mst, user_local_mst.

Q: Can I add a new site to an existing intranet that has shared tables?

A: Yes. See the online help.

Q: Can I add an existing site that is currently on a non-shared intranet to an intranet that has shared tables?

A: Yes.

Q: Can I change an existing site that is currently on a shared table intranet to be on a different intranet?

A: Yes. You would need to unshare the tables for the site's current intranet, which then rebuilds them at all the sites in the intranet. Then you would move the site to another intranet. And then you would reshare the tables in the old intranet.

Q: What happens to users at the other sites if the master site goes down for an extended period?

A: All sites on the intranet that link to the master site will not be able to access/update the tables that are only on the master site.

Q: Do I still need to set up replication categories and replication rules at each site?

A: Yes. Even if you select a category to be shared (controlled by the master site), you still may have to replicate that category, because there may also be other tables (not `_all` or user tables) as well as non-table data, such as stored procedures, included in the category.

Although shared tables still appear in the replication category, those tables are not replicated when a rule for that category is set up between sharing sites. Also, if you try to add a shared table to a new replication category, the system will not allow it unless you are in the master site.

Q: Will my site see data from all other sites on the intranet when we share a table?

A: If the dropdown list in a form is based on an `"_all"` table, and if you are in a using site where that `_all` table is actually a view, then it will access that view.

If you share user tables, an administrator at any site on the intranet can see and update data for all users and groups, if the administrators have the appropriate permissions. If you originally share user tables and then change to per-site user tables, the user and group information for all users on the intranet is loaded into the tables at each non-master site.

Q: When I upgrade to another service pack or version, will the upgrade utility recognize when a site is using a view and not try to update a non-existing `_all` or user table?

A: This depends on how the application handles upgrades.

Q: If I have existing sites containing the same user names and group names or IDs, what happens when I share the user tables?

A: If the system finds matching user names or group names with different IDs in different sites during the sharing process, it assigns new IDs to the names in the shared user tables at the master site. It also updates the IDs in any non-shared tables that reference the IDs at the non-master sites.

However, the processing assumes that you do not have conflicting user names, for example, the user name "jjones" is unique among all users on all sites on the intranet. If you have existing sites with conflicting user names, for example, "jjones" is James Jones at site OH but Jean Jones at

site MI, then you must change one of the user names and all its referenced areas before sharing the user tables.

It is best to share the user tables before sites contain user and group data, if possible. Processing runs much faster with minimal data, and you are less likely to encounter conflicts.

Chapter 12: If Replicating Data, How Often?

12

STOP: Read Appendix A, “Replication Overview,” to get a general understanding about how replication works before you continue. Without that understanding, the replication planning steps probably will not make sense.

Think About This...

Should you use *transactional* or *non-transactional* replication between your sites - or a combination of both?

Replication rules specify *when* and *where* to replicate the categories of data you chose earlier.

Do This...

Update the ReplicationRules Spreadsheet

Indicate which of your rules are transactional or non-transactional. For non-transactional rules, also specify the interval type and timing. For both types, indicate whether to update all columns.

Background Information

Once administrators have defined the source site, target site, categories, and time frames when replication will occur, they can write their replication rules: "Any time this data changes in this place, replicate it to this place." They then regenerate the replication triggers to put the replication process into play.

Replication Rules Form

You will set up the replication rules after logging into Infor Mongoose, on the Replication Rules form at each site. The spreadsheet you create in this step will be the basis for the rules you set up. You can copy and paste rows from the spreadsheet into the grid view of this form.

Interval Type, Interval, and Start Interval At

For each rule, use these three fields together to specify the interval at which replication will occur. For example, you might choose an interval type of Minutes and then enter 90 in the interval field. All interval types except transactional are non-transactional, which means that there is no live connection to another site.

Types are:

- Transactional - Provides a live constant connection from one site to another. Updates are made to the remote site immediately. No interval or start time is required.
- Immediate - Immediately sends the replicated data to the replication queue. (That does not mean the data will appear immediately in the other site. It depends on how soon the queue is retrieved by the other site.) No interval or start time is required.
- Minutes - Replicates data every *nn* minutes, where *nn* is the value specified in the Start Interval field.
- Hours - Replicates data every *nn* hours, where *nn* is the value specified in the Start Interval field.
- Days - Replicates data every *nn* hours, where *nn* is the value specified in the Start Interval field.

Update All Columns

Specify Update All Columns for a rule if you want every table column included in that category/rule to be updated on the target site even if it wasn't changed by a user interaction on the local site. This is *not* recommended for replication, due to the high data traffic it may generate.

If Update All Columns is not specified, replication occurs only for the columns that have been changed.

Rule Execution

Replication happens chronologically, based on the order things happened in the source system. For example, an order line cannot be added until after the order is added. So, when looking at the list of tables in a category, the replication process changes data from the listed tables in the proper order - it does not simply run down the list of tables alphabetically to determine what is different.

Think About This...

Use the application event system to set up and maintain workflow or automated events. For a multi-site system, the following information about events is relevant:

- Event handling is limited to the site(s) contained in the application database associated with the configuration governing the event object.
- The Dispatch IDO Request event action can initiate IDO activities in another site, if it specifies a URL() that controls activities in another site.
- If an event action effects a database change that normally triggers a replication activity, the replication activity will occur when the event action executes.

Do This...

If you are using multiple utility servers, you can create an Events spreadsheet indicating which servers should monitor which application databases. See the information and examples under “Monitoring Events” on page 56.

If you want to replicate event metadata between sites in different application databases, you can create the appropriate custom replication categories and rules to handle it.

Background Information

In its most basic form, the application event system consists of three interrelated components:

- Events are uniquely named incidents that can be triggered during the use of an application. Events can have multiple triggers and can be fired by user actions, conditions in a database, other events, or other situations.

- Event handlers are software routines and data that determine how the system should respond when the associated event is triggered. Each event can have multiple handlers but each handler can be associated with only one event. Each event handler consists of one or more event actions.
- Event actions are the individual tasks or bits of work that are performed by the event handler. Each event handler can have multiple actions.

For example, suppose you have a sales manager who wants to be notified whenever someone adds a new customer to the system. You could use the application event system to set up an event that fires whenever anyone adds a new customer into the system, and use event handlers and actions to automatically generate a notification that is sent to the sales manager.

Events and event handlers are defined for an application database. If you want an event handler to run only at certain sites in a multi-site application database, use the **Applies To Sites** field to specify the sites.

For more information about how the event components work together, see the *Application Event System Guide*.

Monitoring Events

Through the Service Configuration Manager on the utility server, you can configure the Event Service to monitor any known configurations from any and all utility servers. General guidelines:

- Select an optimal combination for best load balancing.
- It is pointless to monitor a configuration whose application database will contain no event handlers or will fire no events. However, note that Infor or another vendor from whom you purchased an add-on product may have included event handlers or code that fires events in their distribution; so you should check for these while considering which configurations to monitor.
- You can create new configurations solely for monitoring by the Event Service.

Each configuration in the monitoring list represents an independent software timer on an independent operating system thread whose time slices can be scheduled (by the operating system) to run on a different CPU than the others, where available.

Example 1

Hardware: One superfast/multiprocessor utility server.

Goal: You want to handle all events on one server.

Implementation: Pick a set of configurations, each of which points to a different application database, and which together span all application databases. Monitor that set.

Details:

Configuration	App database	Monitor?
ILL_Prod	ILL_App	Yes
ILL_Test	ILL_App	No
CAL_Prod	CAL_App	Yes
OH_Prod	OH_App	Yes

Example 2

Data: Six sites (ILL, CAL, OH, COL, CHINA, and FRANCE).

Hardware: Three utility servers, one of which (Util1) is comfortably loaded with other functions (replication, TaskMan, IDO communications, etc.).

Goal: Spread the load from the six site application databases over the two free utility servers (Util2 and Util3).

Implementation: Monitor the largest, heavy-traffic site alone or with another lesser site. Monitor the rest together.

Details:

Server	Configuration	App database	Monitor?
Util2	ILL	ILL_App	Yes
Util2	CAL	CAL_App	Yes
Util3	OH	OH_App	Yes
Util3	COL	COL_App	Yes
Util3	CHINA	CHINA_App	Yes
Util3	FRANCE	FRANCE_App	Yes

Example 3

Data: One particularly heavily used site (ILL_App) with many events firing per second, and/or a need for immediate response to queued and adjourned events.

Hardware: One superfast/quadprocessor utility server (Util1), plus other slower utility servers.

Goal: Select items from the super-site's event queues on as many processors as possible.

Implementation: Monitor the super-site multiple times from the superfast server (once per CPU) and once from the other servers. Monitor the other sites from the other servers, evenly.

Details:

Server	Configuration	App database	Monitor?
Util1	ILL_1	ILL_App	Yes
Util1	ILL_2	ILL_App	Yes
Util1	ILL_3	ILL_App	Yes
Util1	ILL_Prod	ILL_App	Yes
Util2	CAL_Prod	CAL_App	Yes
Util2	OH_Prod	OH_App	Yes
Util2	ILL_Prod	ILL_App	Yes
Util3	ILL_Prod	ILL_App	Yes
Util3	COL_Prod	COL_App	Yes
Util3	CHINA_Prod	CHINA_App	Yes
Util3	FRANCE_Prod	FRANCE_App	Yes

Chapter 14: How Many Licenses (and What Types) Do You Need?

14

Determine which users - and how many users - will need to work in multiple sites. Determine the types of licenses these users require.

You may already have had some discussions with your sales representatives about licensing. This step refines your existing licensing decisions based on the multi-site planning decisions you have made.

Consider the following questions:

- How many people need access to only one site?
- How many people need access to multiple sites?
- Will you have custom forms?
- Do you want to control licensing for all sites on an intranet from a master site?
- Will you use demo and/or pilot databases?

Think About This...

You need a license document to apply your licenses. Infor provides this document in the form of a .txt file when you purchase an Infor product. The file contains, in an encrypted format, all the information needed to apply your licenses to each application database. However, you must determine what licenses you need before the license document can be provided.

Do This...

Create a License Spreadsheet

Review the background information in the *Licensing* document. Then create a License spreadsheet to list the types and number of licenses you need.

Consult your Sales Representative about any changes to your existing license requirements.

Background Information

Review the *Infor Mongoose Licensing* guide for information about how licenses are used in a multi-site environment. This document is available on the Support web site, and it explains the licensing of modules in a multi-site environment, how the intranet licensing option works, and which product and module licenses have special multi-site versions. The guide also includes a multi-site licensing example.

Chapter 15: Is Your Naming Scheme Logical?

15

If you have a large multi-site system, you will have many site IDs, site names, configuration names, intranet names, group names, and database names to manage. Now that you have set up your initial plan, review the flowcharts and spreadsheets for naming consistency.

It will be much easier to keep track of which site you are currently logged into if the names you use are logical, and the configuration name matches the application database name, which matches the site name, and so on.

Think About This...

- Once a Mongoose site is created, changing its site ID is time-consuming and requires a complete stoppage of work in all databases that communicate with, or replicate to, the site whose ID is being changed.

The site ID should remain the same throughout your test, pilot, and production phases, if possible.

- Avoid generic names (for example, "Site1") that do not differentiate your sites. Use names that describe the location, the hierarchy, or what the site is used for.
- Avoid using the following special characters in configuration names and site IDs or site names:
 - \ (back slash)
 - / (forward slash)
 - : (colon)
 - * (asterisk)
 - ? (question mark)
 - " (double quote)
 - ' (single quote)
 - < (left arrow)
 - > (right arrow)
 - | (vertical bar)
 - (embedded space)

- The site ID, the site name, and the application database name should relate to each other, if possible. The site name can be much longer than the ID. For example, if your Site ID is ARIZ, your site name could be Arizona and your application database name could be ARIZ_App.
- See also the information on configuration names on page 28.
- In web server names, avoid using special characters other than '-' or '.' A Microsoft security feature does not preserve session state when web server names in URLs contain other special characters. If your web server must use other special characters, use the IP address in URLs pointing to the server.
- For site groups, use names that define their function (for example, FINANCE) or their location (for example, PACIFIC) - depending on the logical grouping being done.

Do This...

Update Flowchart and Spreadsheets

If you decide to change your naming conventions based on the information in this chapter, update the flowchart and spreadsheets as needed.

Chapter 16: How Easily Will Your Structure Incorporate Later Changes?

16

Now that you have a hierarchical structure of sites, and rules for setting up data flow between them, consider how any radical change to that structure will affect it.

Think About This...

- How will later additions/deletions of sites, or changes to existing sites, affect this structure? Is it set up in a way that will allow changes with the least disruption?
- What if your company buys another company - possibly in a different country? What if your company is acquired? Will this structure be flexible enough to accommodate those changes?
- What if one site has to be taken down for maintenance - will the structure allow your business to continue running during that time?

Do This...

Revise Your Flowcharts and Spreadsheets

If you decide to rearrange your hierarchical structure or data flows after reviewing the questions in this chapter, then update the flowchart and spreadsheets to match.

Appendix A: Replication Overview

A

The information in this appendix is similar to the first chapter in the *Replication Reference* guide, available on our support site. For more details about how replication works, please refer to that guide.

Note: Most of this section is intended for users who have sites in different application databases. If you have all of your sites in one application database, see “Multiple Sites in One Database” on page 70.

About Replication

Data replication is the copying of data between Mongoose sites. The site whose data will be copied is the source, or local, site. The site receiving the copied data is the target, or remote, site.

Replication also allows the system to transmit procedure execution requests (SPs) between the source and target sites.

Infor Mongoose supports two replication methods: transactional and non-transactional.

Transactional Replication

This method (also sometimes referred to as "synchronous" or "connected") performs updates on the target site, based on the transaction being performed at the source site. All changes on both the local database and the target database are committed or rolled back together. Transactional replication assumes that the source site and the target site are always connected through SQL Server and have the same database schema (same Mongoose version).

Transactional replication allows immediate updates between databases without the use of queues or XML-formatted content. Only database triggers and direct stored procedure calls are used to replicate the data.

Data errors (for example, requesting an item that does not exist at the specified remote site) are caught immediately, and the system will not allow the user to save the record with the error.

Transactional replication generally should be used between sites on the same intranet. For best performance, both site databases should be on SQL Server machines within the same LAN, or even on the same SQL Server machine.

Non-Transactional Replication

This method (also sometimes referred to as "asynchronous" or "delayed") assumes that the source site and target site may not be connected through SQL Server. Non-transactional replication uses inbound/outbound queues and XML documents to transfer data or to pass application calls (remote procedure calls, or RPCs).

The sites need not be on the same version of Mongoose; however, if you use different versions, you might need to define XSL transformations so the data arrives at the target site in a usable format for that version. Transformations are required if something significant has changed from one version of the data source to the other, or the target site has more differences than just new nullable columns.

You can also use non-transactional replication to communicate with other applications, if transformations are provided. For more information, see the document *Integrating IDOs with External Applications*.

Non-transactional replication can occur "immediately" (that is, updates are placed in the replication queue as soon as the Replicator service polling interval has elapsed), or at specified intervals (that is, updates are put in the queue when the time interval specified in the replication rule has elapsed). As soon as it reaches the queue, the data is processed by the replication services.

Data errors in non-transactional replication must be handled by the target site system. The user at the source site may not be aware that a replication-related error occurred.

Non-transactional replication populates the ShadowValues table. This and other replication-related tables grow very quickly and may impact system performance.

General Notes About Replication

You can use transactional replication between some of your sites and non-transactional replication between others; it depends on how the sites are connected and how quickly you want the data to be available.

When data is updated in the target system via replication, the replication process bypasses business rule validation and triggers, so replication should only be performed between "trusted" sites. Referential integrity and database-level constraints still apply. Note also that trigger validation is not applied to tables in any case.

_All Tables

The application database can include many tables ending in "_all." In a single-site database, these tables contain data for all sites, while the corresponding base tables contain data only for the local

site. The `_all` tables might include only a subset of the columns from the base table—just enough information to perform local processing on other sites' data.

Example

A system has two sites, OH and MI. OH is replicating its (OH) billing terms data to MI, and MI is replicating its (MI) billing terms data to OH. The OH_App database has a `terms_mst` table with columns and rows similar to this:

terms_code	description	due_days	disc_days	disc_pct	prox_day	tax_disc	cash_only	prox_code
2%	2/10 Net 30	30	10	2	0	0	0	99
2%P	2/10 Proxy 30	0	10	2	30	0	0	99
4%	4/10 Net 30	30	10	4	0	0	0	99
4%P	4/10 Proxy 30	0	10	4	30	0	0	99
5%	5/10 Net 30	30	10	5	0	0	0	99

The MI_App site database has a `terms_mst` table with columns and rows similar to this:

terms_code	description	due_days	disc_days	disc_pct	prox_day	tax_disc	cash_only	prox_code
COD	COD Only	0	0	0	0	0	0	99
N15	Net 15 Days	15	0	0	0	0	0	99
N30	Net 30 Days	30	0	0	0	0	0	99
N45	Net 45 Days	45	0	0	0	0	0	99
NT	No Terms	0	0	0	0	0	0	99
P30	Proxy Day 30	0	0	0	30	0	0	99

Both OH and MI also have a `terms_mst_all` table with columns and rows similar to this:

site_ref	terms_code	description	tax_disc
MI	COD	COD Only	0
MI	N15	Net 15 Days	0
MI	N30	Net 30 Days	0
MI	N45	Net 45 Days	0
MI	NT	No Terms	0
MI	P30	Proxy Day 30	0
OH	2%	2/10 Net 30	0
OH	2%P	2/10 Proxy 30	0
OH	4%	4/10 Net 30	0
OH	4%P	4/10 Proxy 30	0
OH	5%	5/10 Net 30	0

Notice that the `terms_mst_all` table has a `site_ref` column to distinguish the records for each site. It does not include all the columns from the base table - only the ones that typically would be used in a multi-site environment.

When a billing term is added at OH or MI, the `_all` tables at both sites are updated.

General Notes About `_All` Tables

`_All` tables at the local site are populated with local site data through database triggers when the base table is updated. The `_all` table may also contain remote site data, populated through replication (depending on the replication rules defined at the remote sites).

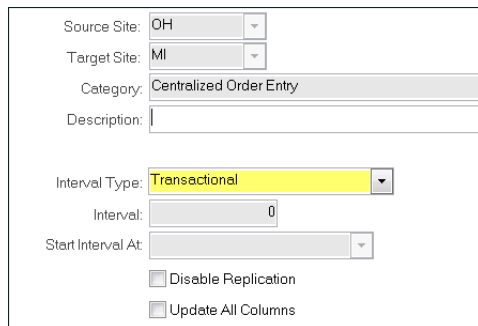
`_All` tables are used when the information in a base table *is not* typically shared among sites (for example, customer orders or transfers). When the information in a base table *is* typically shared among all sites (for example, customer addresses), there may be no need for an `_all` table. Data is replicated directly from the base table in the source site to the base table in the target site. So, such tables usually are replicated directly.

Replication Categories and Rules

Your application based on Mongoose might include default replication categories. You can include these categories in replication rules, or you can create additional categories as needed. Replication categories are defined on the **Replication Categories** form. See your application's documentation for information about any default replication categories that are defined for the application.

How Categories Are Used with Rules

In order to replicate these categories to other sites, you must create one replication rule for each source-target site combination and category. For example, if you want to replicate order entry data from site OH to site MI, and you want to use transactional replication, create a rule in the Replication Rules form at the OH site:



Source Site: OH

Target Site: MI

Category: Centralized Order Entry

Description:

Interval Type: Transactional

Interval: 0

Start Interval At:

☐ Disable Replication

☐ Update All Columns

After the rule is defined and saved, click **Regenerate Replication Triggers** on the Replication Management form. This regenerates the table triggers so that the data is replicated according to the specifications in the category and rule.

In certain cases, replication rules and categories determine which sites are available when performing Mongoose functions.

Point to Point Replication

Replication is point-to-point between sites. There is no "routing forward." For example, assume the following:

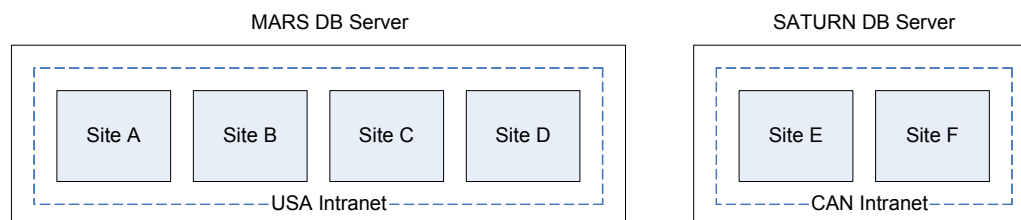
- Site A has a rule to Site B that replicates category X (which includes the table xxx).
- Site B has a rule to Site C that replicates category X.
- Site A does *not* have a replication rule to Site C that replicates category X.

If the xxx table is updated at Site A, the change is replicated to the xxx table at Site B; however, that change is not replicated from Site B to Site C. For an xxx table change at Site A to be replicated at Site C, there must be a replication rule in Site A that sends that data to Site C as well.

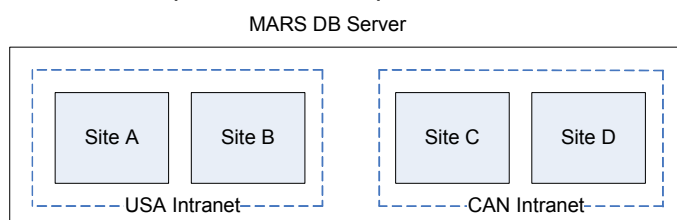
Intranets

When you configure a site, you assign it to an intranet (on the Sites or Sites/Entities form). Intranets are used to segregate a company's sites into virtual groupings that reflect the actual network setup.

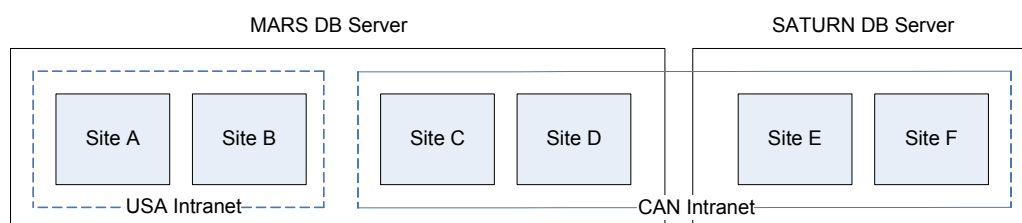
Usually, all the sites on the same LAN and/or database server will belong to the same intranet, and usually sites on different database servers are in different intranets.



However, it is possible to set up sites on the same database server that are in different intranets.



It is also possible to set up one intranet with sites on different servers.



If you have multiple sites in a single application database, and if the sites are sharing `_all` or user tables (through the Intranet Shared Tables or Intranet Shared User Tables utilities), then all sites in that database must be on the same intranet.

Generally, transactional replication should be performed only between sites on the same database server, for performance reasons. However, if your database servers are on a very fast network, transactional replication between sites on different servers is possible.

When a system includes multiple intranets on different servers, the data generally should be transferred between intranets through non-transactional replication (XML documents). When the schema in the source and target databases are not identical—for example, Mongoose to another application that uses Mongoose, or different versions of your application—an XSL transformation (style sheet) should be used. Each intranet has an ASP page that provides intranet access to the inbound queue. The ASP is the gateway to other databases. The address of the ASP is defined on the **Intranets** form.

Transport Protocol

Non-transactional replication between sites on *different* intranets generally uses HTTP POST (internet) communication. Non-transactional replication between sites on the *same* intranet does not use HTTP POST; instead, it is handled directly by one utility server and its MSMQ infrastructure.

When Mongoose communicates with other applications using ION, the transport protocol is ESB. The protocol is specified on the **Intranets** form.

Master Sites and Shared Tables

If your environment has many sites, large amounts of shared data, and many users, you may want to set up one site as the *master site* for an intranet. Master sites are Mongoose sites that control some data for all other sites on an intranet.

If you use a master site, certain tables can reside *only* on the master site and are shared (read and written to through a SQL view) by other sites on the same intranet. No replication needs to occur for the shared tables, which can greatly improve system performance and simplify the setting up of replication rules.

After you set up a master site for an intranet and share tables between the sites on the intranet, the intranet is considered a "shared table" intranet. You can add new sites to a shared table intranet. Removing sites from a shared table intranet is more complex, because you must first unshare the shared tables (that is, recreate them) at the site being removed from the intranet. For more information about sharing and unsharing `_all` and user tables, see the online help.

For more information about setting up master sites and shared tables, see the online help and the *Multi-Site Implementation Guide*.

Multiple Sites in One Database

Replicating `_All` Tables

If all sites in your system are in the same application database, the base tables already contain data for each of the sites, so the `_all` tables are not populated and are simply views to the base tables. In that case, replication of `_all` table data is not necessary.

If at least one site is in a separate database, and you are replicating data to/from that site, the `_all` tables are populated.

Replicating Base Tables

Base tables can include a multi-site column that contains the site name, so that each site in the database can maintain its own records in the table.

You might want to replicate base table data between two sites in the same database. For example, you might want to maintain codes at one site and replicate them to other sites in the database. Whenever a change is made to a code record in the OH site, you want to replicate that change to the MI site. You could set up a replication category that includes the code table, with the Retain Site field cleared. (Clearing the Retain Site field allows the system to create or update a record in the same database table.) Then you create a rule from OH to MI that includes that category.

Replicating Shared Tables

Shared tables have names that do not end in `_all`, and they do not contain a `site_ref` column. For example, the Event table is shared. If all sites in your system are in the same application database, the data for these tables is shared across all sites; that is, the data is not split out by site. In that case, replication of shared table data is not necessary.

If at least one site is in a separate database, and you want that site to "share" data with the other sites, create appropriate replication rules to/from the site.

Glossary

Asynchronous Replication

Another term sometimes used for *non-transactional replication*.

BOD

Business Object Document, an XML document standard defined by Open Applications Group Integration Specifications (OAGIS). This type of document is used when interfacing with many other Infor applications.

Configuration Server

A framework component that dynamically gives the client a URL to use for its session, drawing on a pool of utility servers configured by the system administrator.

Form

In the Mongoose user interface, the equivalent of a screen or a window. See Chapter 4, "Can You Use the Same Forms Database for Multiple Sites?"

Intranet

In Mongoose, represents a logical grouping of sites in your enterprise. An intranet may represent a grouping of Mongoose sites that exist together on a high-speed network, and for which you want common administration. (Common administration can include replication, or creation of master sites.) An intranet may also represent an external non-Mongoose system, allowing you to set up replication rules between Mongoose sites and external systems. See Chapter 8, "How Many Mongoose Intranets Do You Need?"

Intranet Licensing

Optional feature where licensing for all sites on an intranet is maintained in and controlled by the master site for the intranet.

License Module

A license module gives you access to certain specified functionality within Mongoose. You purchase specific license modules based on your company's needs.

Master Site

Mongoose site that controls some data for all other sites on an intranet. For example, master sites are used with the Multi-Site Shared Tables feature. See Chapter 11, "Should Some Data Be Shared Through Master Sites Instead of Replication?"

Multi-Site Table

A database table that includes a site_ref column. Records for various sites are stored in the table, and the value of the site_ref column determines which site the record belongs in.

Non-Transactional Replication

Uses inbound/outbound queues and XML documents to replicate content (data) or to pass application calls (RPCs). Non-transactional replication can occur at any of these intervals: Immediate (which means updates go directly into the replication queue), or at a set number of minutes, hours, or days. Once the data is in the queue, it will be processed by the Replication service when the specified time interval has elapsed. See Chapter 12, "If Replicating Data, How Often?"

Replication

The copying of data between Mongoose sites.

Site

Logically, a site is any place where work is done. Thus, a site can correspond to company headquarters, a manufacturing plant, a distribution center, or a legal company that requires financial reporting. Even if all of these facilities share a single physical building, each can be considered a logical site. See Chapter 2, "How Many Sites Do You Need?"

Stored Procedure

A group of Transact-SQL statements compiled into a single execution plan. Stored procedures are stored in the SQL database and executed upon request - through database triggers or calls from an application.

Synchronous Replication

Another term sometimes used for *transactional replication*.

Token

Allows one user login for a specified license module. Named user and concurrent user licenses are available, and their use of tokens is different. See the *Licensing Guide* for more information.

Transactional Replication

A live constant connection from one site to another. Mongoose replicates the transaction to the remote site as soon as the user at one site saves changes. It assumes that both the source and target sites are always connected through SQL Server and have the same schema. See Chapter 12, "If Replicating Data, How Often?"

Trigger

A special class of stored procedure defined to execute automatically when an UPDATE, INSERT, or DELETE statement is issued against a database table or view. Sites use triggers to enforce business rules automatically when data is modified.

View

A mechanism for gathering elements from SQL tables and assembling them into a virtual table. Generated views simplify database queries by eliminating the need to understand the underlying structure of a database. In Mongoose, views can be used with master sites, allowing other sites in the intranet to maintain data at the master site.

Index

Symbols

_All tables

 Description of 66

_all tables vs. _mst tables 16

A

AccountAuthorizations_mst table 46

ActiveBGTasks_mst table 37

All-in-one database option 19

Application database 10, 19, 23

Application shortcut 29

ApplicationEvent System 55

Applies To Sites field 56

Applies to Sites field 17

ASP 69

authorizations, local or global 46

B

BOD 36

C

Client configuration name 28

ConfigServer.aspx 29

ConfigServerURLs.xml 29

Configuration groups

 connecting client to 31

 Defining in Configuration Manager 31

Configuration Manager utility 29, 31

Configuration name

 Client 28

Configuration Server 33

 designating 30

 use of 29

Configuration wizard 20

Configurations spreadsheet 28

Corporate structure 9

Culture 21

Customization of forms 19

D

Database servers 34

databases, single vs. multiple 13

Dispatch IDO Request event action 55

DMZ servers 30

Domain name 12

E

ESB 70

 protocol 35

ESB transport protocol 35

Event System 55

External intranet 33, 35, 45

F

Financial structure 9

Flowchart 9, 41, 44

FormControl utility 21

Forms customizations 19

Forms database 19

FormSync utility 21

G

GetConfigurations IDO request 31

Group 25

H

Hierarchy 9

HTTP transport protocol 35

https protocol 33

I

- IDO editor forms 23
- IDO Request Service 30
- Immediate replication 66
- Inbound BODs 36
- Infor ESB 70
- Internet, accessing application over 33
- Interval 52
- Intranet licensing 33
- Intranet Licensing, with shared user tables 46
- Intranet Shared Tables 34
- Intranet Shared User Tables 34
- Intranets 33, 44
 - and utility servers 34
 - form 37, 69
 - shared 70

- ION 36

L

- Language ID 21
- License document 59
- License spreadsheet 60
- Licensing
 - intranet 33
- Live links 44
- Load balancing, events 56
- Local Node name 11

M

- Master site 33, 43
 - _all and user tables 70
 - Removing 47
 - SQL clustering 44

- MongooseConfigs.xml 29
- multiple sites in one database 44
- Multi-site groups 25
- multi-site tables 17
- Multi-site vs. multi-warehouse 13

N

- Non-transactional replication 44

O

- Objects database 23
- Outbound BODs 36

P

- Planning worksheet 6
- Process Defaults form 17

R

- Replication categories
 - List of standard 68
- Replication methods defined 65
- Replication of event metadata 55
- Replication queue listener, utility server restriction 34
- Replication Rules form 52
- ReplicationRules spreadsheet 42, 45, 51
- Report server 37
- Rules, overview 68

S

- sa user 12
- Service Configuration Manager 37, 56
- Shared
 - Intranets 70
- Shared Intranet Tables 43
- single database with multi-site 13

Site

- Description 10
- Groups 25, 62
- ID 9, 62
- Name 10, 62
- Type 10

Site spreadsheet 9, 12, 20, 24, 25, 28, 34, 45

site_ref column 17

SQL

- Clustering 11
- clustering 44
- User 12

SQL Server name 11

SSRS subfolders 17

Strings table 20

T

tables

- multi-site 17

TaskMan Path 37

TaskMan, monitoring sites 36

Time zone 11

Tools, list of 5

Transactional replication 44

Transport method 35

Trusted sites 66

U

Update All Columns field 52

URL for web server 33, 62

user authorizations, local or global 46

User interface 20

UserGroupMap_mst table 46

Utility servers 34

Utility servers and intranets 34

W

Warehouses

- vs. multi-site 13

Web client 39

Web server names 62

Web server URL 33

Web servers 34

Worksheet for planning 6

X

XML documents 69

XSL transformation 66, 69